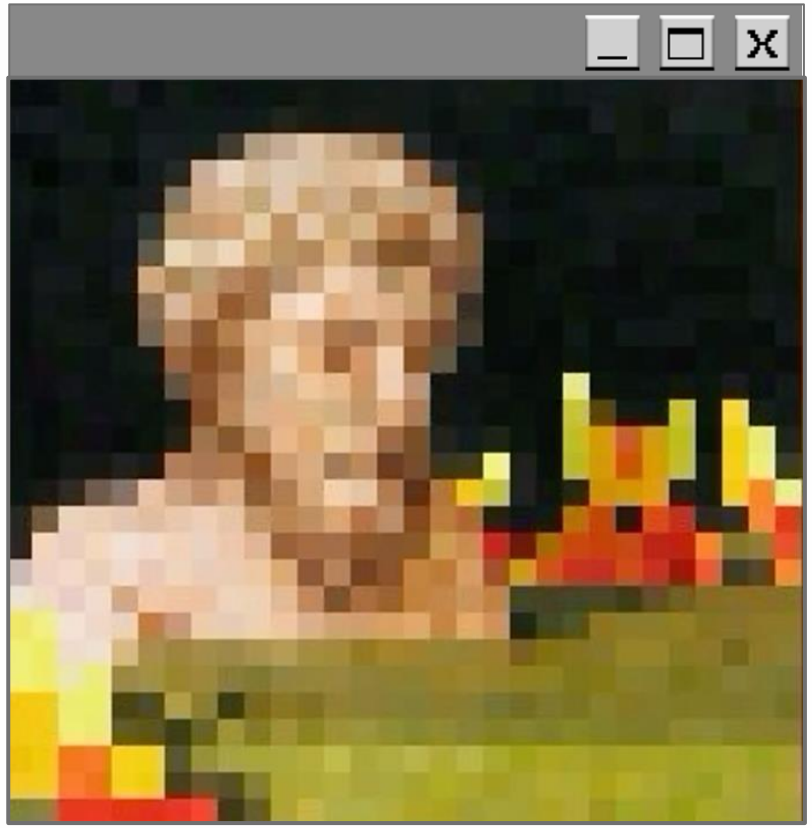
Compresia LZ77 și LZ78



Iacob Alexandru
Profesor coordonator Cristian Rusu

Cuprins

Noțiuni introductive

Compresie = reducerea dimensiunii fișierelor

Compresia este de două tipuri:

- lossless
- lossy

Avantaje: spațiul mai mic de memorie ocupat, transmiterea mai rapidă pe bandă
Dezavantaje: folosirea mai multor resurse pentru comprimare

Glosar de termeni

Redundanță = informație repetată

Entropie = nivelul de informație

Dicționar = structură de date cu intrări de tip *cheie:valoare*

Fereastră glisantă = modalitate de rezolvare a problemelor cu subșiruri

Stream = secvență de date, potențial infinită, care devine valabilă cu timpul (nu toată deodată)

Algoritmul LZ77

Publicat în anul 1977 de Abraham Lempel și Jacob Ziv

Introducerea acesteia a permis procesarea arbitrară a unor porțiuni de date într-un mod optim

Algoritmii de tip LZ77 reușesc să ajungă la compresia datelor prin înlocuirea aparițiilor repetate cu referințe la o copie deja existentă în datele originale. O potrivire va fi codificată prin perechi de numere *lungime (l)* – *distanță (d)*

Pentru a putea face potriviri, programul trebuie să rețină o parte din datele cele mai recente. În practică se reține ultimi k biți

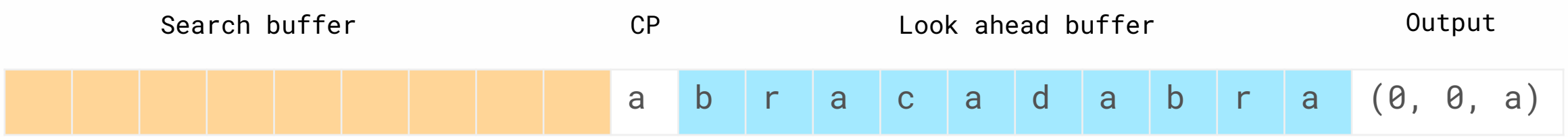
Pseudocod

```
cât timp mai pot să preiau din input execută
|       match := cel mai lung sir din look_ahead_buffer care se repetă în search_buffer
|       dacă match există atunci
|       |       d := distanța cu care trebuie să mă întorc pentru a copia match
|       |       l := lungimea șirului match
|       |       c := caracterul care urmează după șirul copiat
|       |altfel
|       |       d := 0
|       |       l := 0
|       |       c := 0
|       └■

|       output (d,l,c)
|       elimină l+1 caractere de la inceputul ferestrei (search_buffer)
|       s := pop l+1 caractere de la începutul inputului (look_ahead_buffer)
|       adaugă s la finalul search_bufferului

└■
```

Algoritmul LZ77



Algoritmul LZ77

Search buffer									CP	Look ahead buffer											Output
									a	b	r	a	c	a	d	a	b	r	a		(0, 0, a)
								a	b	r	a	c	a	d	a	b	r	a			(0, 0, b)

Algoritmul LZ77

Search buffer									CP	Look ahead buffer											Output
									a	b	r	a	c	a	d	a	b	r	a		(0, 0, a)
								a	b	r	a	c	a	d	a	b	r	a			(0, 0, b)
							a	b	r	a	c	a	d	a	b	r	a				(0, 0, r)

Algoritmul LZ77

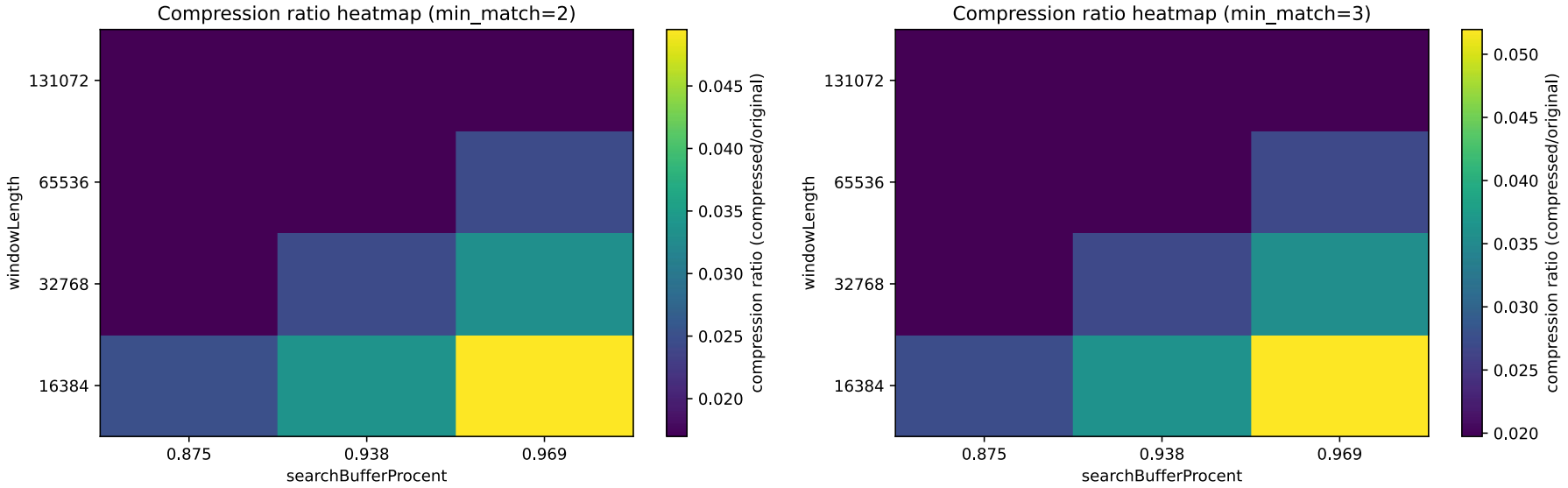
Search buffer									CP	Look ahead buffer											Output
									a	b	r	a	c	a	d	a	b	r	a		(0, 0, a)
								a	b	r	a	c	a	d	a	b	r	a			(0, 0, b)
							a	b	r	a	c	a	d	a	b	r	a				(0, 0, r)
						a	b	r	a	c	a	d	a	b	r	a					(3, 1, c)

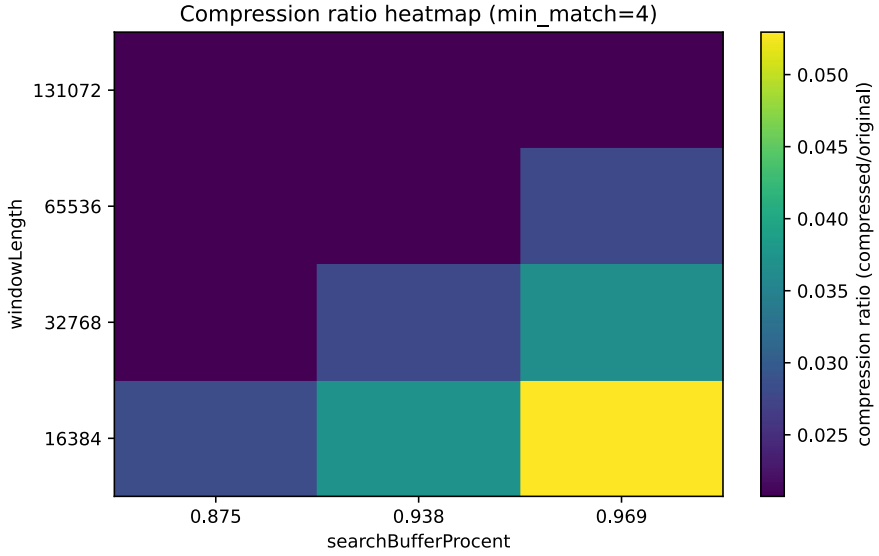
Algoritmul LZ77

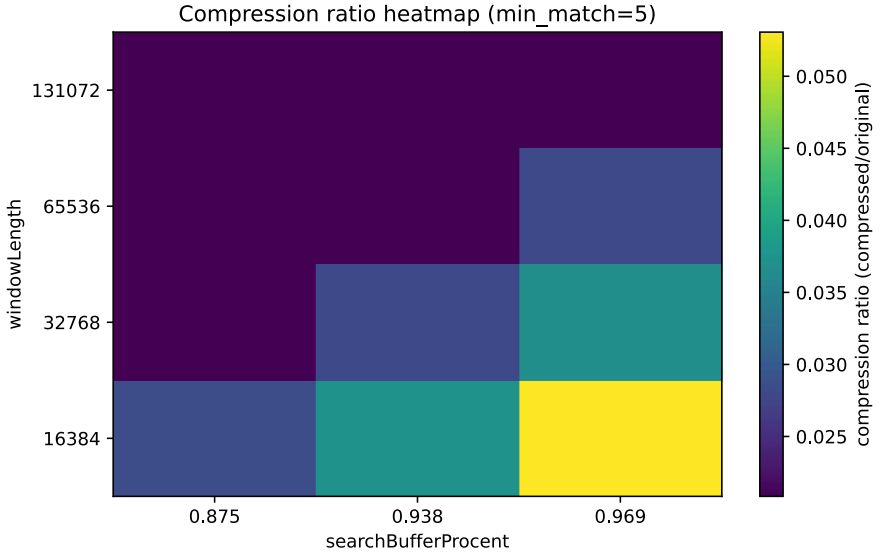
Search buffer									CP	Look ahead buffer											Output
									a	b	r	a	c	a	d	a	b	r	a		(0, 0, a)
								a	b	r	a	c	a	d	a	b	r	a			(0, 0, b)
							a	b	r	a	c	a	d	a	b	r	a				(0, 0, r)
						a	b	r	a	c	a	d	a	b	r	a					(3, 1, c)
				a	b	r	a	c	a	d	a	b	r	a							(2, 1, d)

Algoritmul LZ77

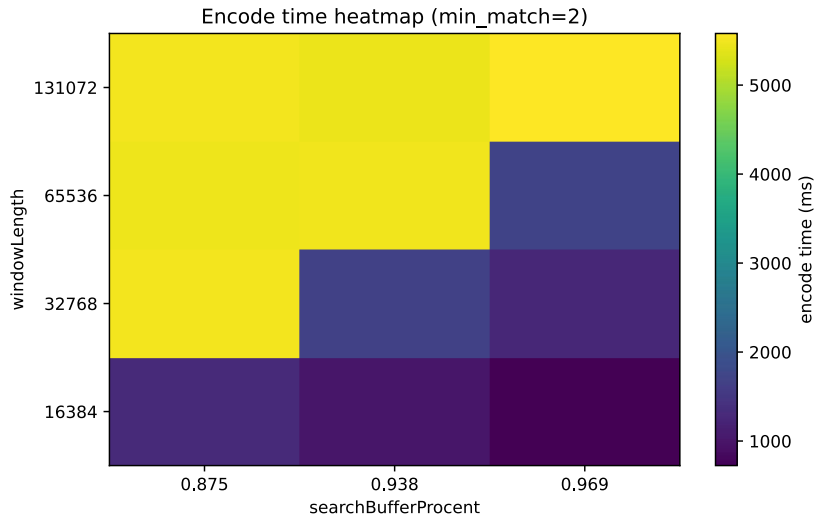
Search buffer									CP	Look ahead buffer											Output
									a	b	r	a	c	a	d	a	b	r	a		(0, 0, a)
								a	b	r	a	c	a	d	a	b	r	a			(0, 0, b)
							a	b	r	a	c	a	d	a	b	r	a				(0, 0, r)
						a	b	r	a	c	a	d	a	b	r	a					(3, 1, c)
				a	b	r	a	c	a	d	a	b	r	a							(2, 1, d)
		a	b	r	a	c	a	d	a	b	r	a									(7, 4, -)

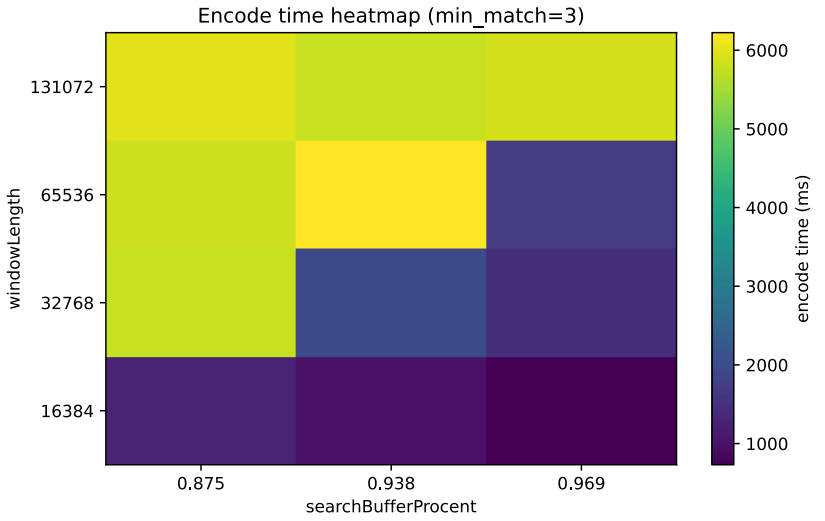


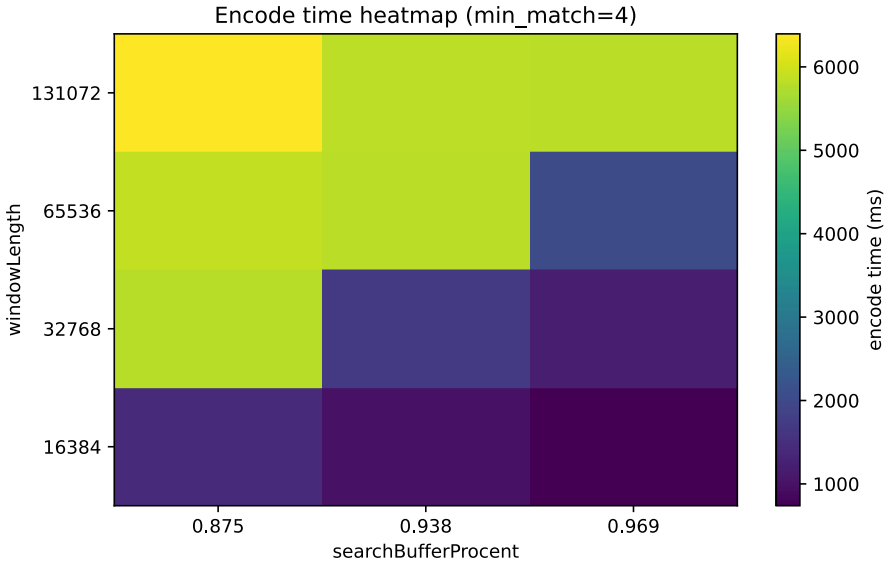


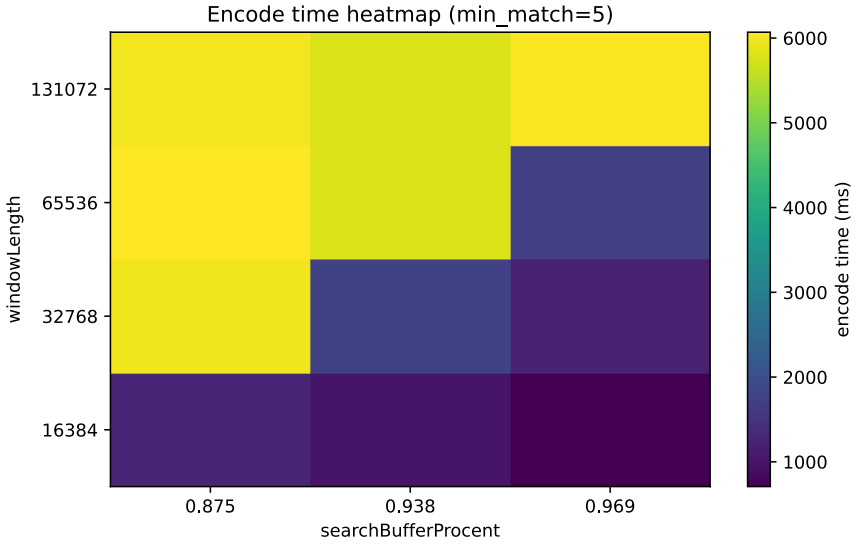


Comparație rata de compresie pentru diverse valori ale lui *min_match*

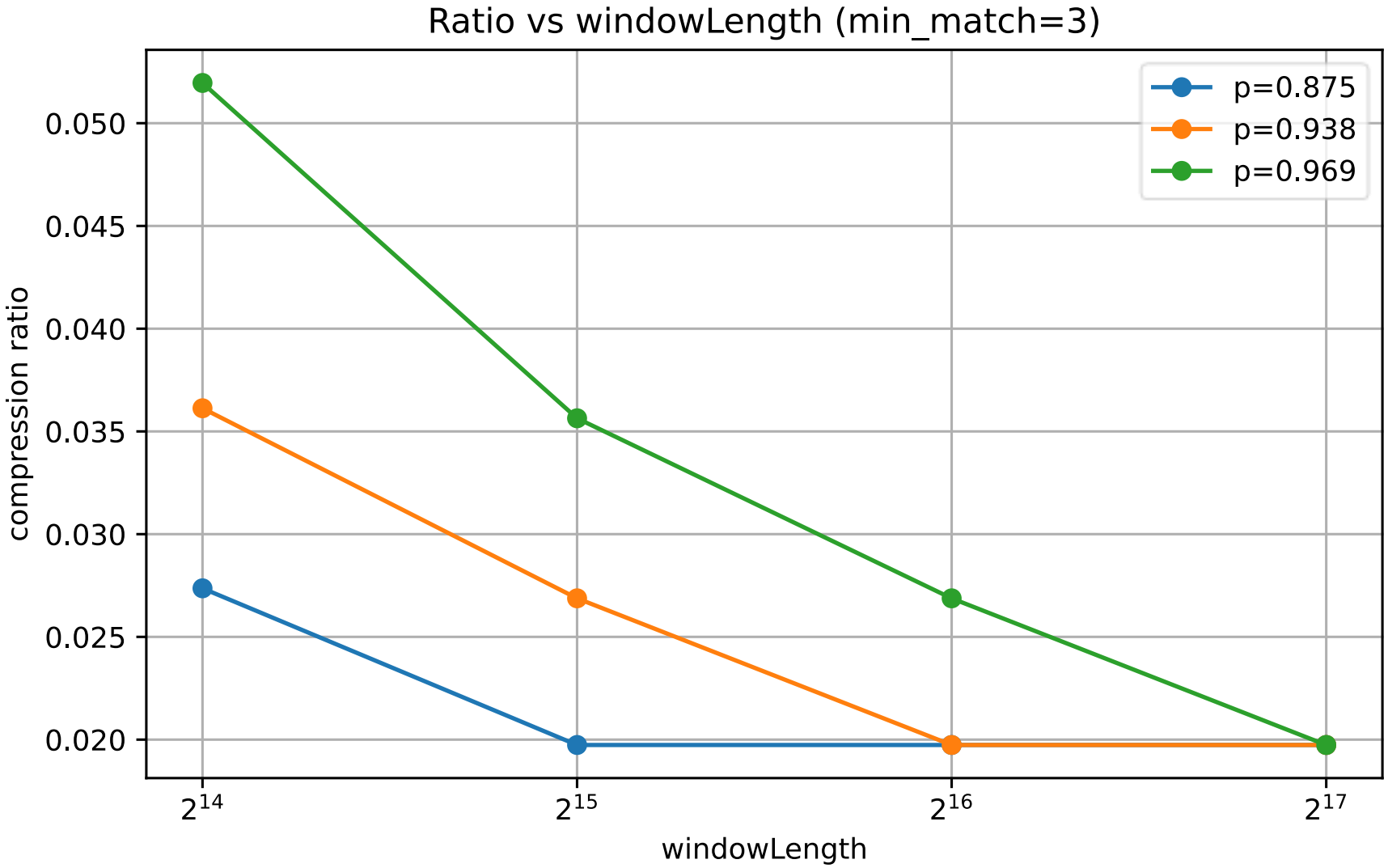




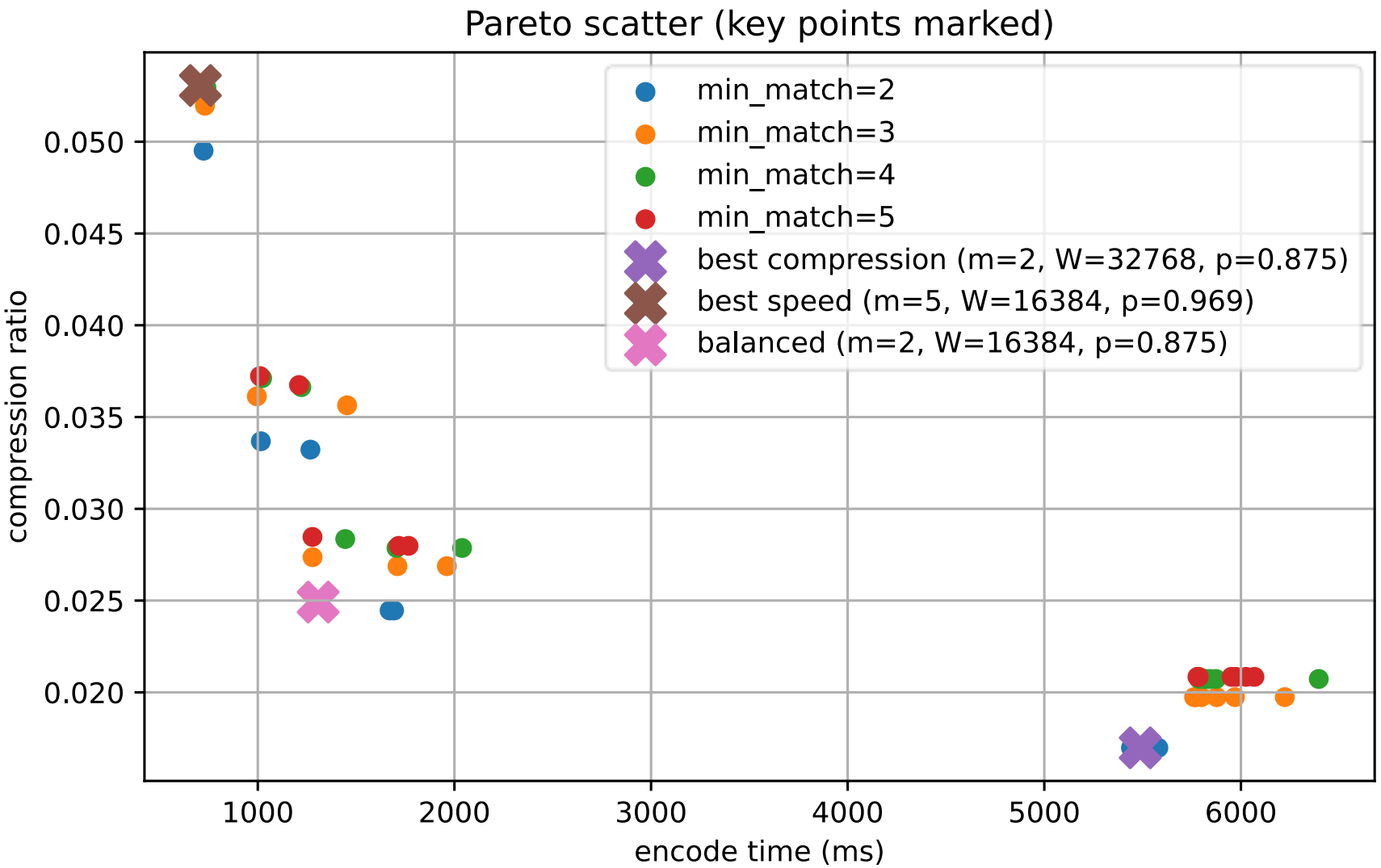




Comparație timpului de codare pentru fiecare *min_match = 2..5*



Rata de compresie în funcție de *windowLength*



Pareto scatter pentru detectarea soluției optime

Algoritmii LZ78

Publicat în anul 1978 de Abraham Lempel și Jacob Ziv

Algoritmii LZ78 de compresie se bazează pe crearea/existența unui dicționar de tokenuri.

Fiecare intrare din dicționar este de tipul *dicționar[...] = {indice, token}*, unde:

- indicele reprezintă o secvență deja găsită
- tokenul este următorul caracter.

Pseudocod

```
input ← citește fișierul ca string
input ← input + ‚$’
tree ← dicționar gol
index ← 1
output ← listă cu (None, ‚’)
i ← 0
cât timp i < len(input) execută
|     parent ← 0
|     cât timp tree conține cheia parent execută
|     |     găsit ← FALS
|     |     pentru fiecare (child, token) din tree[parent] execută
|     |     |     dacă token = input[i] atunci
|     |     |     |     parent ← child
|     |     |     |     i ← i+
|     |     |     |     găsit ← ADEVĂRAT
|     |     |     |     break
|     |     |     ⌊■
|     |     ⌊■
|     |     dacă găsit = FALS atunci
|     |     |     adaugă (index, input[i])
|     |     |     break
|     |     ⌊■
|     ⌊■
|     dacă tree NU conține cheia parent atunci
|     |     tree[parent] ← [(index, input[i])]
|     ⌊■
|     adaugă (parent, input[i]) la output
|     index ← index+1
|     i ← i+1
⌊■
return output
```

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7		
1	(0, A)	A	8		
2	(0, B)	B	9		
3	(0, R)	R	10		
4			11		
5					
6					

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7		
1	(0, A)	A	8		
2	(0, B)	B	9		
3	(0, R)	R	10		
4	(1, C)	AC	11		
5					
6					

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7		
1	(0, A)	A	8		
2	(0, B)	B	9		
3	(0, R)	R	10		
4	(1, C)	AC	11		
5	(1, D)	AD			
6					

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7		
1	(0, A)	A	8		
2	(0, B)	B	9		
3	(0, R)	R	10		
4	(1, C)	AC	11		
5	(1, D)	AD			
6	(1, B)	AB			

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7	(3, A)	RA
1	(0, A)	A	8		
2	(0, B)	B	9		
3	(0, R)	R	10		
4	(1, C)	AC	11		
5	(1, D)	AD			
6	(1, B)	AB			

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7	(3, A)	RA
1	(0, A)	A	8	(7, B)	RAB
2	(0, B)	B	9		
3	(0, R)	R	10		
4	(1, C)	AC	11		
5	(1, D)	AD			
6	(1, B)	AB			

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7	(3, A)	RA
1	(0, A)	A	8	(7, B)	RAB
2	(0, B)	B	9	(1, R)	AR
3	(0, R)	R	10		
4	(1, C)	AC	11		
5	(1, D)	AD			
6	(1, B)	AB			

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7	(3, A)	RA
1	(0, A)	A	8	(7, B)	RAB
2	(0, B)	B	9	(1, R)	AR
3	(0, R)	R	10	(6, A)	ABA
4	(1, C)	AC	11		
5	(1, D)	AD			
6	(1, B)	AB			

Algoritmul LZ78

Exemplu: ABRACADABRARABARABARA

Index	Output	Entry	Index	Output	Entry
0	-	Ø	7	(3, A)	RA
1	(0, A)	A	8	(7, B)	RAB
2	(0, B)	B	9	(1, R)	AR
3	(0, R)	R	10	(6, A)	ABA
4	(1, C)	AC	11	(7, **EOF**)	RA **EOF**
5	(1, D)	AD			
6	(1, B)	AB			

Fișierele testate

01% Text repetitiv (repeat.txt)

```
AAAAABBBBBCCCCCDDDDDEEEEE
AAAAABBBBBCCCCCDDDDDEEEEE
AAAAABBBBBCCCCCDDDDDEEEEE
AAAAABBBBBCCCCCDDDDDEEEEE
AAAAABBBBBCCCCCDDDDDEEEEE
AAAAABBBBBCCCCCDDDDDEEEEE etc.
```

02% Text natural (natural.txt)

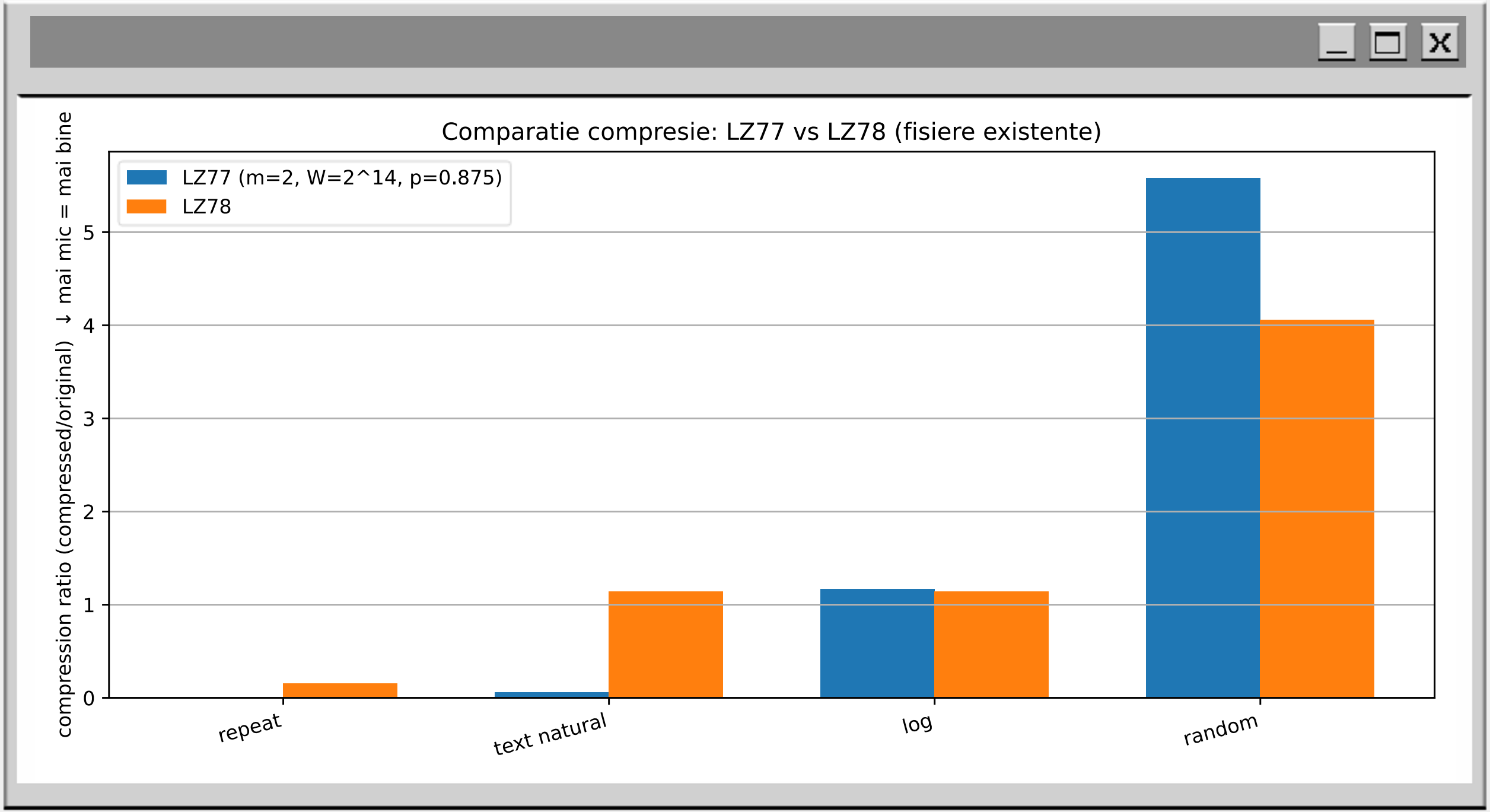
```
Astazi am testat mai multe
   configuratii pentru compresie si am
   notat rezultatele in mod sistematic.
   Algoritmii LZ cauta repetitii
   locale, iar in texte reale repetitia
   apare in cuvinte etc.
```

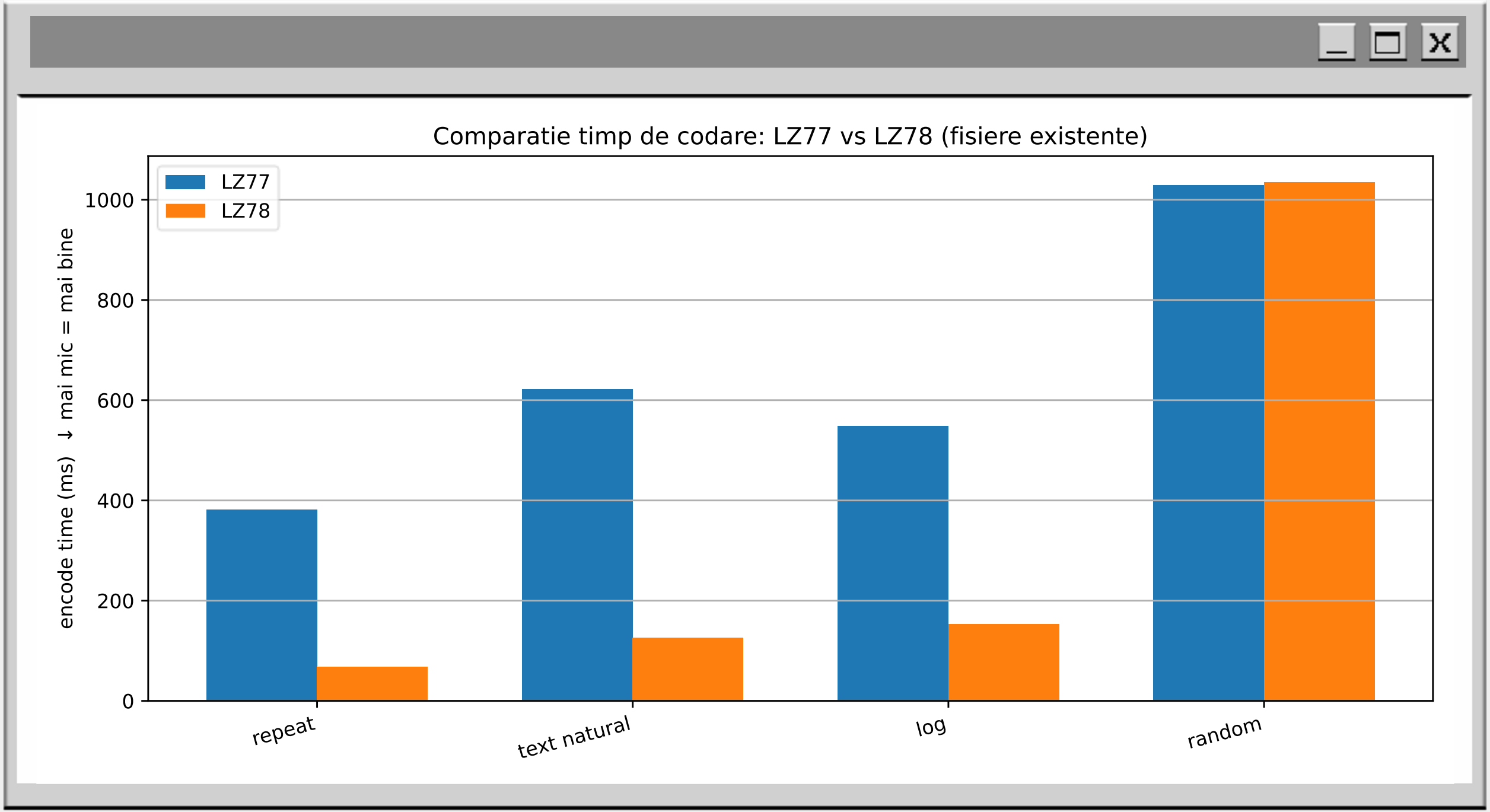
03% Text de tip logs (log.txt)

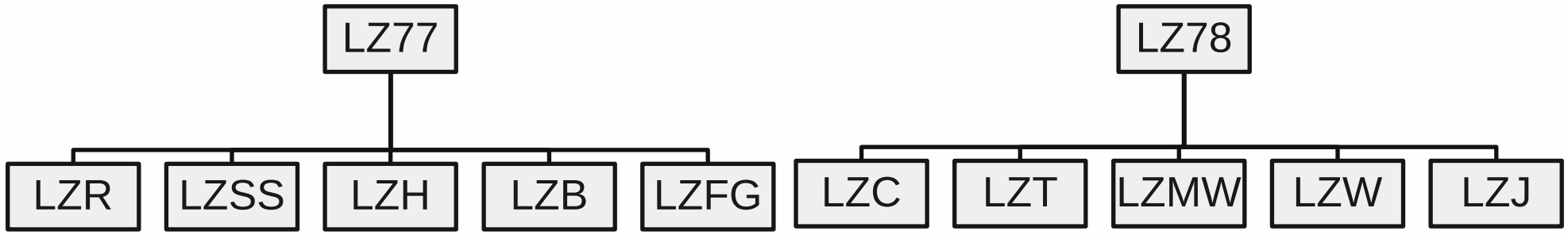
```
2026-02-03
   10:00:00,user000,LOGIN,item0000,0.00,OK,ses
   sion=0000,ip=192.168.0.0012026-02-03
   10:00:01,user001,LOGOUT,item0001,0.10,OK,se
   ssion=0001,ip=192.168.1.0022026-02-03
   10:00:02,user002,VIEW,item0002,0.20,OK,sess
   ion=0002,ip=192.168.2.003 etc.
```

04% Text random (random.txt)

```
{od&JFCr&n\&.l-2edlBD (d;z$<@.1C5-
   Ja=u@2RJtB=RnlWmTS(Hf^6pWk,L][U!y#i
   f>D|LkDmWJ6{UuVTA/I@'}j({v^Fu7WI{";
   C'P|hDeOZIiB:$OB<
   Y}6sHrF*;H*?2?ZUCr
   l+got]u'2_iXW_7G,b'%o’ etc.
```


Comparatie compresie: LZ77 vs LZ78 (fisiere existente)
LZ77 (m=2, W=2^14, p=0.875)
LZ78
compression ratio (compressed/original) ↓ mai mic = mai bine
0
1
2
3
4
5
repeat
text natural
log
random


Comparatie timp de codare: LZ77 vs LZ78 (fisiere existente)
LZ77
LZ78
encode time (ms) ↓ mai mic = mai bine
0
200
400
600
800
1000
repeat
text natural
log
random



Aplicații
- ZIP
- GZIP
- STACKER

Aplicații
- GIF
- V.42
- COMPRESS

Familia de algoritmi
LZ77 și LZ78

Bibliografie

- CHANG, E., FERNANDO, U., & HU, J. (n.d.). Lossless Data Compression. Retrieved from Data Compression: A General Study: https://cs.stanford.edu/people/eroberts/courses/soco/projects/data-compression/lossless/lz78/concept.htm
- COLLINS, G., & SYME, D. (1995). A theory of finite maps. Higher Order Logic Theorem Proving and Its Applications.
- JAGADISH H. PUJAR, L. M. (2010). A NEW LOSSLESS METHOD OF IMAGE COMPRESSION AND DECOMPRESSION USING HUFFMAN CODING TECHNIQUES. Journal of Theoretical and Applied Information Technology.
- KIMURA, N., & LATIFI, S. (2005, April 4). A Survey on Data Compression in Wireless Sensor Networks. Proceedings of the International Conference on Information Technology: Coding and Computing (ITCC'05).
- MAHDI, O. A., MOHAMMED, M. A., & MOHAMMED, A. J. (2012). Implementing a Novel Approach an Convert Audio Compression to Text Coding via Hybrid Technique. IJCSI International Journal of Computer Science Issues.
- SHANNON, C. (1948). A Mathematical Theory of Communication. Bell System Technical Journal.
- ZIV, J., & LEMPEL, A. (1977). A Universal Algorithm for Sequential Data Compression. IEEE Transactions on Information Theory.
- ZIV, J., & LEMPEL, A. (1978). Compression of individual sequences via variable-rate coding. IEEE Transactions on Information Theory.